

Lab Maintenance Operations Guide

Management overview and technical reference

Compton College

Windows endpoint maintenance, telemetry, and approval-gated repair

Document baseline: Package 3.0.3, GitHub commit c82b57e, published October 8, 2026. Covers the active numbered scripts, deployment and support utilities, configured task times, and the new Script 08 n8n repair-approval process.

Prepared for management, systems engineering, network/security staff, and desktop support. Read the executive summary and operating overview for the service model; use the script chapters and runbooks for implementation and troubleshooting.

What this document establishes

- What each script is intended to accomplish and which settings or files it changes.
- When the current package schedules work, and how those start times differ from completion times.
- How persistent Windows corruption becomes an approval request and when an approved endpoint may proceed.
- How technicians verify outcomes, interpret telemetry, and identify configuration gaps before fleet rollout.

Internal technical reference. Contains site-specific hostnames, share paths, and the supplied workflow image. Credential values are omitted.

Reviewed source is authoritative for this guide. The live deployment share, installed endpoint versions, and n8n configuration require their own verification.

Executive summary

The Compton College maintenance program automates recurring care of shared Windows lab computers. It refreshes an approved script package, prepares Windows Update, removes eligible local profiles, standardizes applications and user settings, installs approved updates, coordinates reboots, checks Windows integrity, and records a final health inventory. Most recurring work runs as SYSTEM during Sunday maintenance; time synchronization and Deep Freeze status checks have separate schedules.

What the college gains

- **Consistent lab experience:** verified printing, browser, Office, startup, and application settings reduce workstation-to-workstation variation.
- **Less repeated manual work:** detected update and servicing problems receive defined repair attempts, with reusable logs rather than undocumented technician actions.
- **Measurable outcomes:** execution records, update results, health findings, software inventory, and repair fingerprints support fleet reporting and targeted follow-up.
- **Controlled escalation:** routine DISM/SFC repair remains automatic when enabled, while source-WIM and possible in-place Windows repair require a matching, unexpired n8n approval.

What changed on October 8

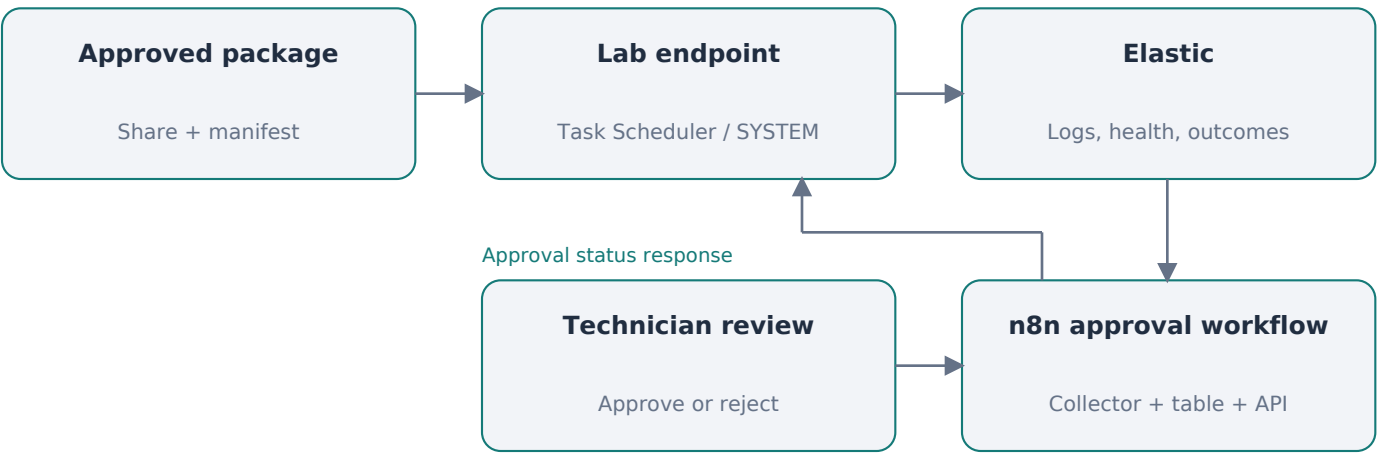
Script 08 now looks up approval using an external key file and validates the computer, corruption fingerprint, status, and expiration. Missing or unavailable approval defers escalation. The scheduled launcher enables the required repair flags, and task registration can update only an existing repair action without changing its schedule. The workflow provides a technician dashboard, decision audit fields, and an authenticated lookup endpoint.

Approval is permission for a later eligible run. Clicking Approve does not initiate Windows Setup. The endpoint first rechecks corruption, validates approval, and attempts the helper's source-repair sequence. An in-place install is a conditional final escalation, not the guaranteed next action.

Current assurance and remaining validation

A synthetic request, dashboard decision, Approved lookup, test-row deletion, and subsequent NotFound lookup were demonstrated. Shared-key retrieval and API lookup were tested under SYSTEM. The GitHub package and changed-file checksums were verified. A real corruption-to-Setup lifecycle and final Windows validation of the merged launcher/registration remain outstanding. This guide records source behavior, not proof that every lab endpoint is running that version.

High-level operating model



Normal maintenance continues; approval gates only persistent-corruption escalation.

The endpoint does the maintenance

Windows Task Scheduler starts a fixed approved action under SYSTEM. The launcher maps that action to a local script, evaluates any configured maintenance policy, and uses a shared execution lock. The numbered script detects conditions, applies its enabled changes, verifies the result, and writes logs and structured records. The central share distributes code; GitHub is the source/version reference, not an endpoint execution service.

Elastic supplies evidence; n8n supplies a decision

Elastic ingestion makes endpoint events available for reporting. For persistent Script 08 corruption, n8n collects approval-required events into a table, renders pending requests, validates technician decisions, and answers endpoint status queries. n8n does not push Windows Setup to the computer. Each endpoint checks approval during its own repair execution.

Three types of work

Type	Examples	Expected outcome
Recurring maintenance	Scripts 00-10, 14, and 16	Updated package, standard configuration, servicing, and verified health.
Conditional repair	Script 08 escalation; Edge repair helper	Work runs only when detection and applicable controls permit it.
Administrative support	Post-deployment, registration, manifest, tests	A technician prepares, validates, or reconciles the managed environment.

In this guide, a scheduled start time is a trigger, a policy window is an authorization condition evaluated by a launcher, and a completion time is an observed outcome. They are different facts.

Contents and reading paths

Executive summary	2
High-level operating model	3
Contents and reading paths	4
Maintenance schedule	6
Schedule behavior and execution controls	7
Script 00 - package synchronization	8
Script 01 - prepare Windows servicing	9
Script 02 - profile cleanup and user settings	10
Script 03 - application updates	11
Script 04 - consolidated lab maintenance	12
Script 04 - changes, targeting, and verification	13
Script 05 - HP and Dell maintenance	14
Script 06 - Windows Update passes	15
Script 07 - coordinated reboot cycle	16
Script 08 - diagnostics and ordinary repair	17
Script 08 - controls, cleanup, and evidence	18
Script 08 - file operations and retention	19
Script 09 - control background updating	20
Script 10 - time synchronization	21
Script 14 - authoritative endpoint inventory	22
Script 14 - changes, findings, and software state	23
Script 16 - Deep Freeze status	24

Deployment tool - Post-Deployment.ps1	25
Support tools - launcher and task registration	26
Support modules - shared framework and Copilot	27
Policies, manifests, and administrative utilities	28
Validation tools and conditional Edge repair	29
In-place repair - management decision model	30
n8n workflow - supplied implementation image	31
n8n - branch-by-branch technical flow	32
Approval table - identity, state, and expiry	33
Approval API and endpoint authorization contract	34
Component-store helper - stages and safeguards	35
Post-Setup verification and return to service	36
Technician runbook - review and approve	37
Technical runbook - deployment and validation	38
Troubleshooting the approval workflow	39
Telemetry, reporting, and operating ownership	40
Retired scripts and replacement ownership	41
Glossary and interpretation guide	42
Source baseline and validation record	43
Source inventory - numbered and deployment scripts	44
Source inventory - support and reference files	45

Maintenance schedule

These are the Sunday triggers defined by Register-Tasks_SYSTEM.ps1 v4.12.0. Times use each endpoint's configured local Windows time zone. For Compton lab systems configured to Pacific Time, interpret the table as Pacific local time. Daylight saving follows Windows configuration. This table does not assert the current exported tasks on every endpoint.

Time	Task	Script / function
01:00	01. Check for Updated Scripts	00 - synchronize approved package
01:15	02. Enable Windows Update Services	01 - prepare servicing environment
01:30	03. Remove User Profiles Weekly	02 - eligible profile cleanup / UI
02:00	04. Sunday Lab Application Maintenance	04 - consolidated application/configuration work
03:15	05. Weekend Apps Update	03 - WinGet and Office servicing
04:15	06. Weekend HP Drivers Update	05 - vendor-aware safe updates
05:15	07. Weekend Windows Updates - 1st Pass	06 - first Windows Update pass
06:15	08. Force Reboot Install Updates	07 - reboot cycle coordination
06:45	09. Weekend Windows Updates - 2nd Pass	06 - second pass after servicing/reboot
07:45	10. Disable Windows Update Services	09 - post-maintenance update control
08:00	11. System Repair	08 - detect, repair, and approval-gated escalation
09:00	12. Weekly Endpoint Health Inventory	14 - final health/software evidence

Additional and conditional triggers

Trigger	Action
Daily 00:00, 04:00, 08:00, 12:00, 16:00, 20:00	Script 10 time synchronization.
System startup	Script 07 -StartupResume, for an active reboot cycle.
Monday 07:00	Script 16 Deep Freeze status; StartWhenAvailable enabled.
After an in-place repair is staged	Helper verification task: startup delayed 20 minutes; additional retry trigger.
n8n Schedule Trigger	Collect approval requests; exact deployed interval was not supplied.
Manual / approved conditional action	Post-deployment, administrative tests, Edge repair; no recurring registration here.

Schedule behavior and execution controls

Order is intentional; completion is not chained

Script 04 precedes general application and Windows updating so its restore point is created earlier in the cycle. Script 14 runs last to summarize the resulting maintenance state. The tasks are independent fixed-time triggers. A slow earlier task can overlap a later trigger; the launcher lock has a 120-second timeout and does not guarantee that a later task will wait until its predecessor finishes. Use durations and task results to identify missed or delayed stages.

Task settings in the current registration code

Managed tasks use SYSTEM / ServiceAccount with highest privileges, permit battery operation, have a 12-hour execution limit, ignore a new instance of the same task while one is active, and request two restart attempts at five-minute intervals. The ordinary weekly tasks use StartWhenAvailable=false. The Monday Deep Freeze task explicitly enables catch-up. These settings describe registration intent; retry behavior also depends on Task Scheduler and the script exit result.

Launcher and policy behavior

Full reconciliation registers fixed ActionId invocations instead of accepting arbitrary script paths. Policy entries for Scripts 01, 02, 03, 05, 06, 07, 08, 09, and 14 enforce Sunday 00:00-10:00. Script 10 has no enforced window. The reviewed policy does not include the consolidated Script 04 name; the framework currently allows a script with no enforced entry. Its scheduled trigger still places it on Sunday, but that is not an equivalent policy restriction.

Legacy repair-task schedule

The endpoint tested on October 8 had a task named 12. System Repair. The current full package names the repair task 11. System Repair at 08:00. The targeted -SystemRepairOnly option accepts exactly one of those names and changes only its action. It preserves triggers, principal, settings, and enabled state. It does not migrate the older schedule or start the repair.

Entry point matters. A targeted direct repair action bypasses the common launcher's policy-window and global-lock checks. Script 08 still enforces its own approval gate, but direct/manual execution should be scheduled deliberately. The full managed rotation continues using the launcher.

Operational verification

Export or inspect installed task triggers, actions, principal, LastRunTime, LastTaskResult, and enabled state. Compare local script versions and the deployed manifest with the approved baseline. Check the live n8n Schedule Trigger separately; the screenshot establishes workflow structure, not its polling interval.

Script 00 - package synchronization

00_Update-Scripts-FromShare.ps1 | Version 4.11.2 | Source [S01]

Purpose and operational outcome

Keeps C:\Scripts aligned with the approved central package. This is the distribution control that makes a reviewed change available to lab endpoints; editing GitHub alone does not deploy it.

How the script works

1. Resolve the preferred and fallback deployment roots, retaining previously working production roots across self-updates.
2. Validate the source layout and manifest. Restrict installation to the internal approved-file catalog; compare SHA-256 values and parse PowerShell files.
3. Create rollback copies, replace changed files, and install critical supplemental files during catalog/manifest transitions.
4. Retire superseded numbered files only after the replacement Script 04 validates; update/relaunch the updater when required, then invoke task reconciliation.

Changes and fixes it makes

Refreshes scripts, framework modules, policy files, the software-inventory policy, and approved repair helpers. Reconciles recurring tasks. Prevents updater downgrades and avoids replacing production source settings with public placeholders.

Controls and important defaults

Sunday 01:00 through the launcher. Also bootstrapped by Script 01 and used during deployment. Files need accessible source roots, valid manifest entries, matching hashes, and valid syntax. Rollback retention is later managed by Script 08.

How to verify the result

Review per-file updated/unchanged/failed results, source selection, parser/hash errors, updater version, and reconciliation status. Compare C:\Scripts to DeploymentManifest.json; a GitHub commit by itself is not deployment evidence.

Script 01 - prepare Windows servicing

01_Enable_Windows_Update_Services.ps1 | Version 2.8.5 | Source [S02]

Purpose and operational outcome

Restores the service and policy conditions needed for the weekly Windows Update cycle. It also confirms that the master updater task matches the approved schedule; current per-user interface-setting enforcement is documented under Script 02.

How the script works

1. Refresh/run the current updater before importing framework dependencies, so older endpoints can bootstrap an incomplete package.
2. Capture service and conflicting policy state. Restore startup behavior and start Windows Update, BITS, Delivery Optimization, Update Orchestrator, Windows Update Medic, Cryptographic Services, and Windows Installer as applicable.
3. Enable required update tasks and remove conflicting automatic-update policy values; restore NoAutoUpdate=0 and AUOptions=3 for the maintenance servicing state.
4. Retry required service recovery up to four total attempts. If critical services remain unavailable, request a controlled reboot and record the reason.

Changes and fixes it makes

Changes service startup/runtime state, selected update task state, and registry update-policy values. Repairs the Sunday 01:00 updater task when its managed properties drift. Reverses the update restrictions applied later by Script 09.

Controls and important defaults

Sunday 01:15. Requires Administrator or SYSTEM and usable deployment/framework sources. Service shutdown is bounded and may force-terminate an unresponsive service host; this is a servicing operation with real process impact.

How to verify the result

Review before/after service snapshots, service-recovery attempts, policy conflicts, bootstrap failures, task reconciliation, and RebootInitiated/RebootReason. A reboot request is a corrective step; verify recovery after the next boot.

Script 02 - profile cleanup and user settings

02_Remove_User_Profiles.ps1 | Version 2.5.0 | Source [S03]

Purpose and operational outcome

Removes eligible local user profiles to recover storage and reset shared-computer user state. It also applies the standard Windows 11 UI preferences to accessible existing user hives and Default User.

How the script works

1. Enumerate Win32_UserProfile instances beneath C:\Users, collect last-use/creation information, and measure eligible profile sizes.
2. Exclude the configured accounts, special/system profiles, and loaded profiles. Apply the optional age filter only when OlderThanDays is greater than zero.
3. Remove associated OneDrive tasks and profile registration, then delete profile contents through bounded parallel cleanup with resumable state.
4. Record completed, failed, timed-out, and deferred work; apply verified user-environment settings to available current, loaded, offline, and default hives.

Changes and fixes it makes

Deletes eligible profiles and their local data, not just temporary cache. Sets left taskbar alignment, hides Widgets/Task View, shows the full search box, enables the classic context menu, disables Search Highlights, opens Explorer to This PC, and shows extensions/hidden files. Teams/Chat is left unchanged. Version 2.5.0 moves Copilot and Edge InPrivate ownership out of this script to Script 04.

Controls and important defaults

Sunday 01:30. Default exclusions include Default, Default User, Public, All Users, MISAdmin, CC-Student, and dswaney. SkipLoadedProfiles and SkipSpecialProfiles are true. OlderThanDays defaults to 0: there is no age requirement in the scheduled invocation. Two deletion jobs, a 40-minute overall budget, and a 15-minute per-folder job limit are configured.

How to verify the result

Review deleted and skipped identities, reasons, measured profile size removed, actual disk-free-space change, timeouts, and resumable cleanup state. Confirm that protected or currently loaded users were skipped; recovered bytes and profile-count metrics answer different questions.

Script 03 - application updates

03_Weekend_Apps_Update.ps1 | Version 2.2.0 | Source [S04]

Purpose and operational outcome

Maintains applications that WinGet can service and records available, completed, failed, and deferred upgrades. It also services installed Office Click-to-Run and produces application inventory for maintenance planning.

How the script works

1. Find a usable WinGet executable, initialize network/tooling settings, refresh sources, and capture available upgrades, including evidence of pinned/deferred packages.
2. Run the general upgrade operation with the configured source and version options.
3. Retry suitable explicit/remaining package targets and update Office Click-to-Run with a bounded wait.
4. Capture final installed versions and remaining upgrades, compare results, and publish per-application telemetry and a latest inventory file.

Changes and fixes it makes

Updates supported installed applications and Office servicing components. Captures version transitions and remaining upgrade candidates. This does not guarantee coverage for every product or supply a custom updater for every academic application.

Controls and important defaults

Sunday 03:15. IncludeUnknown=true; IncludePinned=false; AttemptMSStore=false; UpdateOffice=true; Office wait is 30 minutes. Microsoft.AppInstaller is deliberately deferred from unattended targeted retries because it contains the running WinGet tooling. Honor package pins and source/license restrictions.

How to verify the result

Use before/after versions, updated count, failed/remaining items, PinnedOrDeferred details, and 03_Weekend_Apps_Update.app-inventory.latest.json. An upgrade command accepting work is weaker evidence than a verified version transition.

Script 04 - consolidated lab maintenance

04_Sunday_Lab_Application_Maintenance.ps1 | Version 1.17.0 | Source [S05]

Purpose and execution model

Runs the recurring application and lab-configuration work previously spread across Scripts 11, 12, 13, 15, 17, 18, and 19. It executes at Sunday 02:00. Embedded source blocks are readable text, staged to a per-run folder, and run in isolated 64-bit Windows PowerShell child processes where appropriate. A child's functions, strict mode, or exit statement cannot silently terminate all later sections. The runner aggregates section outcomes and propagates meaningful child failure/reboot evidence.

Section in execution order	Current source setting / outcome
1. System Restore	Enabled. Enable protection, initialize VSS-related services, create/verify a managed restore point, retain managed history.
2. SHARP / PaperCut / printer	Enabled. Detect installed driver/client state and maintain StudentSecurePrint connectivity.
3. Office LTSC 2024	Enabled. Detect products, conditionally migrate supported suites, install and verify activation.
4. Edge InPrivate	Enabled for SSB-122-*, SSB-114*, SSB-171*. Autologon configuration is explicitly false.
5. Elastic Agent	Disabled: RunElasticAgent=false. Embedded install/enroll/health logic remains available.
6. Browser policies	Enabled. Compton homepage/startup, Chrome prompt controls, default-browser association policy.
7. Adobe local PDF policy	Enabled. Suppress account/cloud/trial experiences while retaining local PDF functions.
8. Honorlock	Disabled. Group Policy owns extension deployment.
9. Stellarium Location Services	Enabled for SSC-216* and MS-203*.
10. Startup allowlist	Enabled. Disable non-allowlisted startup entries; suppress HP notification popups.
11. Final Copilot cleanup	Enabled. Shared-module cleanup and policy reapplication after this runner's sections.

Source setting takes precedence over feature descriptions. The current GitHub copy retains Elastic installation code and prefix configuration but does not execute that section. The live protected-share copy may be configured differently; verify its switch and enrollment configuration.

Script 04 - changes, targeting, and verification

04_Sunday_Lab_Application_Maintenance.ps1 | Version 1.17.0 | Source [S05]

Restore point, printing, and Office

System Restore enables OS-volume protection where supported, adjusts creation-frequency handling, initializes shadow-copy services, creates a restore point, verifies it exists, and prunes obsolete points belonging to its managed description. It is a recovery checkpoint, not a full disk-image backup. Printing uses driver-package identity/version evidence, installer state, and connection verification to avoid unnecessary reinstalls. A timed-out printer connection must be reviewed independently from a successful driver or PaperCut installation.

Office inventory distinguishes Office 2024, older supported perpetual/LTSC products, and Microsoft 365 Apps. Older Click-to-Run products are removed with targeted Office Deployment Tool configuration. MSI migration requires a supported RemoveMSI configuration. Microsoft 365 replacement is permitted only when Deep Freeze installation is detected; non-Deep-Freeze computers retain Microsoft 365 and record a skip. Installer files are staged locally, installation and machine-wide activation are checked, history is persisted, and the local stage is cleaned. Office installation detection does not imply activation success.

User experience and configuration

Browser policies set <https://www.compton.edu> in Chrome, Edge, and Firefox. Chrome sign-in, startup profile selection, onboarding, and default-browser prompts are suppressed. A default-associations XML specifies Chrome for HTTP, HTTPS, .htm, and .html. Targeted Edge startup uses InPrivate. It intentionally leaves existing Winlogon/autologon settings unchanged; the presence of an older embedded autologon implementation does not mean it runs.

Adobe policy suppresses Pro trials, upsells, sign-in, Document Cloud/connectors, and online first-run experiences while preserving local viewing, printing, forms, and signatures. Stellarium targets enable Windows Location Services; that section is configuration, not a guarantee that Stellarium itself is installed or updated.

Startup and Copilot controls

Allowed startup items are DWRCST.EXE, initialise.bat, OneDrive.exe, student.exe, teacher.exe, and RtkAudUService64.exe. Other discovered entries are disabled rather than uninstalled. HP consumer notification processes/tasks are narrowly suppressed without removing HP hardware-support components. Copilot cleanup imports Maintenance.Copilot.psm1 and reports packaged/desktop results and remaining artifacts.

Target and result evidence

Elastic's configured StartsWith prefixes are IB1, IB2, SSC, AHB, SSB, MS, and VT; wildcard characters are not used in that list. Honorlock's retained target list is SSC-216* and AHB-146* but its section is off. Read section Success, ExitCode, RebootRequired, SkippedDisabled, failure details, Office migration/activation fields, printer state, and verified registry settings. The runner's overall result should not hide a failed child.

Script 05 - HP and Dell maintenance

05_Weekend_HP_Drivers_Update.ps1 | Version 3.4.3 | Source [S06]

Purpose and operational outcome

Selects the correct vendor workflow, maintains supported drivers and utilities, and records hardware/driver changes. It also applies selected lab power and wake settings and removes orphaned HP support components from Dell images.

How the script works

1. Detect the vendor and capture hardware, storage, Device Manager, driver, and prior-baseline information.
2. On HP, resolve HPIA and HP CMSL prerequisites. On Dell, resolve Dell Command Update, its service, and the .NET Desktop Runtime using configured share/Internet fallbacks.
3. Scan candidates, filter unsafe unattended categories, and install the permitted selection. Apply supported HP BIOS network-update controls, AV scheduled power-on, and Dell wired Wake-on-LAN/adaptor wake verification.
4. Capture final driver versions and change evidence; clean eligible orphaned HP management/support applications, services, tasks, and inactive component drivers on Dell while preserving active peripheral support.

Changes and fixes it makes

Installs allowed vendor updates; changes selected BIOS power/network-update and Ethernet wake settings where supported; cleans vendor-image leftovers. Reports vendor utility failures, update selection, per-driver results, and baseline changes.

Controls and important defaults

Sunday 04:15. IncludeBIOS and All are not supplied by the normal scheduled action, so BIOS/firmware and boot-sensitive storage/chipset/controller/RST/VMD/NVMe categories are not broadly enabled. Explicit broad switches require separate review. AV power-on state targets midnight on Sunday and Monday-Friday in this version; do not infer a 07:00 setting from older discussions.

How to verify the result

Check installed/skipped/failed candidates, HP/Dell scan/apply exits, driver differences, AV power-on verification, Dell wired WOL BIOS/adaptor state, and baseline evidence. A BIOS WOL setting does not establish that classroom broadcast routing or the wake sender works.

Script 06 - Windows Update passes

06_Weekend_Windows_Updates.ps1 | Version 2.1.10 | Source [S07]

Purpose and operational outcome

Installs available Windows/Microsoft updates, measures update compliance, and records whether servicing requires a reboot. The same script runs twice to accommodate updates that become applicable after earlier servicing or a reboot.

How the script works

1. Ensure NuGet and PSWindowsUpdate prerequisites, then capture build, installed-update, available-update, and pending-reboot state.
2. Optionally reset update components when explicitly requested; record service, folder, and registry outcomes.
3. Install available Microsoft updates within the configured operation timeout, normalizing update identifiers and result properties for telemetry.
4. Rescan and compare installed, failed, remaining, skipped/deferred, and reboot state against the initial snapshot.

Changes and fixes it makes

Installs applicable update payloads and updates Windows servicing state. With `ResetWUComponentsFirst`, also resets selected update services/cache/registration state. It records reboot requirements without initiating the reboot itself.

Controls and important defaults

Sunday 05:15 and 06:45. `ResetWUComponentsFirst=false` by default. `OperationTimeoutSeconds=1800` (30 minutes). Script 01 prepares service access and Script 07 owns reboot coordination. A timeout or failed prerequisite must not be counted as full patch compliance.

How to verify the result

Inspect KB/update identities, installed/failed results, remaining candidates, before/after build state, module version, and pending reboot. Correlate first and second pass `RunIds` rather than assuming the second task always follows a completed first pass.

Script 07 - coordinated reboot cycle

07_Force_Reboot_Install_Updates.ps1 | Version 2.1.1 | Source [S08]

Purpose and operational outcome

Completes servicing that needs restarts while preventing unbounded or falsely counted reboot cycles. A persistent state file and boot-time comparison connect scheduled and startup-resume executions.

How the script works

1. Capture current boot time, logged-on users, update state, and pending-reboot flags/cause details. Acquire the script's single-instance mutex and read cycle state.
2. For an existing stage, require a newer boot time before advancing. StartupResume exits safely when no cycle is active.
3. Request the appropriate reboot only when needed; publish completed logs/state before shutdown. Continue through a maximum of three reboot stages, followed by verification.
4. Report persistent flags with likely causes. A stale non-startup cycle may be recovered after the configured age; startup resume still requires proof of the prior reboot.

Changes and fixes it makes

Writes durable reboot-cycle state and can restart the computer. Optional cleanup of approved stale flags is separate from forcibly clearing unknown servicing evidence. Records whether flags existed before work, whether reboot was issued, and whether it was observed.

Controls and important defaults

Sunday 06:15 plus the registered startup-resume task. RebootDelaySeconds=30; StartupSettlingSeconds=120; AbandonedCycleMinutes=10. MaintenanceCloseout and FinalVerification exist as alternate modes but are not additional recurring tasks in this registration. Mode switches are mutually exclusive; unsafe flag cleanup is not enabled by default.

How to verify the result

Verify stage, current/previous boot timestamps, RebootIssued/RebootVerified, cycle recovery, remaining flags, and persistent-after-third-reboot evidence. An active stage without a newer boot is not permission to count another completed reboot.

Script 08 - diagnostics and ordinary repair

08_System_Repair.ps1 | Version 4.9.2 | Source [S09]

Purpose and execution

Script 08 is the broad endpoint repair and cleanup owner. In the managed launcher, Sunday 08:00 uses AutoRepairOnDetection, AllowComponentStoreEscalation, and AllowCopilotRemoval. The first enables detected ordinary repairs; the second permits escalation only if the n8n gate also passes; the third preserves the existing Copilot cleanup opt-in. A plain invocation defaults AutoRepairOnDetection to false, but independently enabled actions and cleanup can still change the machine.

Area	What it detects or does
Windows servicing	DISM component-store corruption; SFC integrity violations; CBS corrupt-file and driver evidence; online repair and repeat verification.
Storage and volume state	Free space, largest directories, volume scan results, SpotFix eligibility, SMART/NVMe reliability, predicted failure and wear evidence.
Management / platform	Action1 presence/install validation, WMI repository consistency, scheduled-task health, pending reboot, Defender status.
RPC and events	Local endpoint mapper, Event Log/WMI access, remote RPC context, unexpected shutdown evidence, and root-cause classification.
Explorer / shell / Search	Crash and hang evidence, shell extensions, network-path dependencies, cache state, and guarded enabled remediation.
Cleanup	Safe temporary/cache targets, SoftwareDistribution leftovers, log/archive consolidation, telemetry rotation, HP BIOS staging, and updater rollback retention.

Detection-first repair sequence

1. Run diagnostic checks and collect the pre-repair state. Cleanup and validation actions have their own controls.
2. When ordinary autorepair is enabled, attempt DISM repair and the SFC/CBS workflow for detected corruption. HP driver-only remediation needs its independent opt-in and supporting evidence.
3. Repeat DISM/SFC verification. Clean results end this escalation path. Persistent corruption generates a fingerprint and an approval lookup.
4. Perform other eligible detected repairs, such as WMI salvage or volume SpotFix, only within their controls. Preserve detailed outcomes rather than treating one failure as the result of every check.

The n8n approval applies to persistent-corruption component-store escalation. It is not approval for every unrelated optional network, firewall, offline-volume, or HP driver operation.

Script 08 - controls, cleanup, and evidence

08_System_Repair.ps1 | Version 4.9.2 | Source [S09]

Control	Default / significance
AutoRepairOnDetection	False for a plain invocation; true in the managed repair action.
AllowComponentStoreEscalation	False by default; managed action sets true, but matching unexpired n8n approval is still required.
AllowWmiRepair	True, used for a detected inconsistency within ordinary autorepair.
Explorer/Search health repair	Enabled independent switches; Explorer cache cleanup/Search restart can run even without ordinary autorepair.
Network / update / firewall reset	Independent opt-ins, false by default.
Offline volume fix / HP driver repair	Independent opt-ins, false by default.
Aggressive cleanup / event-log clear	False by default; retained diagnostic/event evidence is not intentionally cleared by those optional switches.
AutoRebootIfNeeded	False by default; helper Windows Setup can still perform its own required reboot sequence.

Cleanup ownership

The script consolidates historical maintenance logs and uses LastWriteTime for log aging, prunes expired archive contents, rotates oversized NDJSON, removes expired Deep Freeze JSON events after 14 days, and removes updater rollback folders older than seven days. It cleans approved contents under C:\Temp while preserving the directory and guards dedicated HP BIOS staging removal. Other configured cache targets include Windows temporary/update/CBS/prefetch and report-queue locations. Reparse points and access-blocked protected targets are intentional skips. A cleanup skip is not evidence of storage failure.

SoftwareDistribution cleanup has resumable state, bounded concurrency, a 40-minute overall cleanup budget, and a 15-minute folder-job timeout. The active Script 08 runtime log is staged beneath ProgramData rather than C:\Temp, so its own temporary cleanup does not delete the active log. Cleanup policy is not a substitute for a backup or forensic-retention policy.

Outcome interpretation

Use detailed step results, pre/post corruption flags, repair attempts, disk changes, pending-reboot reasons, storage risk indicators, and the latest JSON/YAML/text log. InPlaceRepairAuthorized starts false. Missing approval records DeferredAwaitingApproval with ComponentStoreEscalationAttempted=false. StartedRebootRequired means Setup handoff, not completed recovery. Failed, CompletedWithCorruption, and Succeeded evidence must be interpreted with the helper's later logs.

Script 08 - file operations and retention

08_System_Repair.ps1 | Version 4.9.2 | Source [S09]

Cleanup runs independently of ordinary autorepair

The normal execution path invokes cleanup and diagnostic helpers even when `AutoRepairOnDetection` is false. Explorer diagnostics also invoke cache cleanup and Windows Search restart when their independent enabled switches permit it. Thus a detection-only invocation is not a read-only assessment. The affected Explorer cache paths use the running account environment; `SYSTEM` context does not automatically mean every interactive user cache is processed.

Target / operation	Reviewed behavior
C:\Temp and approved cache locations	Clean configured contents while preserving container directories; skip protected/reparsed targets where guarded.
Explorer caches	Clear eligible icon/thumbnail and Quick Access destination caches for the running account; optionally restart Windows Search.
C:\Windows.old	Attempt forced removal whenever the directory exists; not gated by <code>AutoRepairOnDetection</code> , <code>AggressiveCleanup</code> , an age test, or <code>n8n</code> approval.
Updater rollback folders	Remove folders older than seven days beneath <code>C:\Scripts\Rollback</code> .
Deep Freeze event files	Prune JSON events older than 14 days beneath <code>C:\Logs\DeepFreeze</code> .
Active maintenance NDJSON	Rotate at the Script 08 caller threshold of 25 MB.
Weekly text/YAML/rotated NDJSON	Archive previous Sunday-to-Saturday loose files to a verified ZIP; exclude active Script 08 log/YAML/NDJSON. Delete eligible loose/Old Logs files and managed ZIPs older than the two-Sundays-ago cutoff.

Windows.old impact

The routine `Windows.old` cleanup takes ownership, grants `SYSTEM` access to that directory, clears attributes, and attempts native recursive removal, then verifies absence. It is distinct from the external approval key file, whose permissions are never changed. Removing `Windows.old` can remove prior-installation recovery material. Preserve required recovery evidence before a later Script 08 run; the current approval gate does not govern this cleanup.

Verify the actual result

Inspect `WindowsOldCleanup`, deleted/skipped target details, archive-entry verification, cleanup errors, rotation results, and disk-free-space change. Local cleanup windows differ from centralized Elastic retention. Confirm logs were collected before relying on local history to investigate an older failure.

Script 09 - control background updating

09_Disable_Windows_Update_Services.ps1 | Version 2.0.7 | Source [S10]

Purpose and operational outcome

Reapplies the lab's post-maintenance Windows Update control state. Its counterpart is Script 01, which restores servicing access at the start of the next weekly cycle.

How the script works

1. Capture selected service and policy state. Stop/control wuauserv, BITS unless preserved, Delivery Optimization, Update Orchestrator, and Windows Update Medic as applicable.
2. Set the selected service startup states, tolerating protected-service limitations according to the script's verification rules.
3. Unless skipped, apply NoAutoUpdate=1 / AUOptions=1 and disable selected Windows Update, Update Orchestrator, and Medic scheduled tasks.
4. Verify the resulting services, task states, and registry values and publish before/after evidence.

Changes and fixes it makes

Changes runtime/startup state of selected update services, disables specific update tasks, and applies automatic-update policy. Version 2.0.7 treats stopped Delivery Optimization as successfully controlled even when Windows retains Auto/Manual startup, avoiding a false error.

Controls and important defaults

Sunday 07:45. KeepBITSAvailable, SkipScheduledTasks, and SkipUpdatePolicy are optional and are not supplied by the normal task. Disabling BITS can affect other software that uses it. Because System Repair runs later, source-WIM repair and any Windows Update fallback must handle the servicing prerequisites deliberately.

How to verify the result

Review services after control, protected-service results, update-task states, and policy values. Check the next Script 01 run restores them. A stopped service here is expected policy state, not automatically a health incident.

Script 10 - time synchronization

10_Sync_System_Time.ps1 | Version 2.0.10 | Source [S11]

Purpose and operational outcome

Keeps the Windows clock synchronized for authentication, certificate validation, event correlation, and reliable maintenance timestamps. It selects the appropriate time source for domain-joined versus standalone systems.

How the script works

1. In Auto mode, select the Active Directory domain hierarchy for domain members or configured manual NTP peers for workgroup computers.
2. Configure Windows Time and its startup/source settings, then restart/start it with bounded recovery and shared-process protections.
3. Request rediscovery/resynchronization with retries.
4. Read the final source, synchronization age, stratum, and clock-offset evidence; record configured changes and warnings.

Changes and fixes it makes

Changes Windows Time service and synchronization configuration, requests clock correction, and reports drift/staleness. It does not repair domain trust merely by correcting the clock.

Controls and important defaults

Daily at 00:00, 04:00, 08:00, 12:00, 16:00, and 20:00. No enforced maintenance window. SyncMode=Auto; manual peer time.windows.com,0x9; three resync attempts; 30-second service-stop timeout; maximum sync age 24 hours; offset alert threshold 5,000 milliseconds. Verify the final source matches the endpoint's domain state.

How to verify the result

Use time_synchronization telemetry, synchronization source/age, offset and command results. A running w32time service alone does not establish a successful or recent synchronization.

Script 14 - authoritative endpoint inventory

14_Endpoint_Health_Inventory.ps1 | Version 1.9.0 | Source [S12]

Purpose and collection model

Produces the weekly endpoint.health snapshot at Sunday 09:00 after the scheduled maintenance stages. Individual collectors are isolated so a failed warranty/network/event query can be recorded while the rest of the inventory is preserved. It is principally a measurement and evidence script, but it also configures restricted WinRM and writes a local Autopilot hardware-hash file.

Domain	Collected evidence and use
Identity / hardware	Hostname-derived building/lab/device identity, manufacturer/model, serial, BIOS/baseboard/enclosure, CPU/memory, monitors, audio, optical drives, printers. Supports locating and comparing endpoints.
OS / performance / storage	Windows version/activation, uptime, reboot state, memory/load, disk capacity/free space, reliability/wear and device problems. Supports health triage.
Security / network	Defender, firewall, BitLocker, TPM, Secure Boot, interfaces and diagnostics, time source, management agents and services. Supports configuration/compliance follow-up.
Servicing / failures	Windows Update state, task health, installed driver/BIOS currency, application crashes/hangs, WER and Windows failure evidence. Supports repeat-detection repair candidates.
Software inventory	Normalized non-system product records, versions, publishers, categories, stable identity and removal tombstones. Supports latest-state software reporting.
Vendor lifecycle	HP/Dell BIOS settings with sensitive values redacted, update evidence, warranty/entitlements and persistent serial-bound snapshots.

Outputs

Writes Maintenance-Telemetry.ndjson, 14_Endpoint_Health_Inventory.latest.json, local inventory snapshots beneath C:\ProgramData\Compton\Inventory, and an Autopilot CSV beneath C:\ProgramData\Compton\Autopilot. Warning/Critical findings produce endpoint.health.finding events. Tracked products produce endpoint.software events; eligible stable failure signatures produce remediation-candidate events.

What it does not prove

A candidate is an input for a controlled remediation workflow, not proof that repair ran. Inventory coverage depends on collector permissions, vendor tools, network/API availability, policy, and freshness. A local hardware hash is not Intune enrollment, and a warranty snapshot is not automatically refreshed on every run.

Script 14 - changes, findings, and software state

14_Endpoint_Health_Inventory.ps1 | Version 1.9.0 | Source [S12]

Restricted remote access and Autopilot preparation

For an AD-joined endpoint on a DomainAuthenticated network, the WinRM collector enables/configures remoting, restricts built-in PowerShell session endpoints to SYSTEM, local Administrators, and resolved Domain Admins, disables Basic authentication and unencrypted service traffic, and manages domain-profile exposure. It applies changes idempotently and consolidates a required WinRM restart. Workgroup/no-domain-profile computers skip this configuration rather than exposing the same listener broadly. The Autopilot CSV is generated locally with restricted file access; no automatic upload or enrollment is implemented here.

Reliable software history

SoftwareInventory.Policy.json controls tracking, exclusions, canonical names, and categories. This version tracks all non-system applications by default, excludes updates/system/runtime components, and does not include loaded-user registry inventory by default. It avoids Win32_Product and its MSI consistency-check side effects. Local snapshots are compared only when computer/policy identity is compatible. A package absent from a complete comparable snapshot emits a removal tombstone with removal time/reason and prior RunId; do not infer uninstallation from an incomplete collector run.

Health signatures and noise reduction

Stable SHA-256 fingerprints omit volatile timestamps/process/report IDs where appropriate. WER records are classified and deduplicated so Windows servicing failures, genuine application crashes, and Edge/Copilot installer crashpad evidence are distinguishable. Access-denied inspection of CapabilityAccessManager.db-wal is recorded as an informational limitation. Repeated USB Code 43 evidence can become a controlled driver-remediation candidate rather than an immediate blind reinstall.

Warranty and BIOS evidence

HP saves the first successful serial-bound warranty result; Sunday first-time lookups are distributed through a hostname-derived window of up to 60 minutes. Refresh is explicit. Dell uses external machine-level TechDirect client credentials and a persistent serial-bound snapshot; credentials are excluded from logs and snapshots. BIOS settings are normalized and password-related values redacted. Optional approved BIOS/driver comparison uses EndpointHealth.Baseline.json; without it, age/currency evidence remains but approved-version compliance is limited.

Verification

Check snapshot RunId/time, failed collector fields, severity totals, specific findings/fingerprints, serial-to-host identity, software tombstones and policy version, local hash-file status, and remote-access changes. Inventory can extend well beyond its 09:00 start because of collector work and warranty staggering; the policy window check occurs before launch, not continuously during collection.

Script 16 - Deep Freeze status

16_Check_Deep_Freeze_Status.ps1 | Version 1.3.0 | Source [S13]

Purpose and operational outcome

Measures whether Deep Freeze is installed and whether the current workstation state is Frozen, Thawed, or Unknown. It supplies the latest-state dashboard with thaw duration and state transitions.

How the script works

1. Locate DFC.exe and determine whether Deep Freeze is installed. Query and normalize the CLI state/output when available.
2. Read the locally retained state history and compare the new observation with the previous state.
3. Preserve the first observation of the current thawed state; calculate ThawedSinceUtc, ThawedForDays/Hours, PreviousState, and StateChanged.
4. Write an atomic per-run JSON event, shared NDJSON telemetry, and latest-event data. Systems without Deep Freeze explicitly publish Installed=false / NotInstalled.

Changes and fixes it makes

Writes state-tracking and event files. It does not thaw/refreeze the workstation, manage secure-channel passwords, or repair domain trust. Explicit NotInstalled evidence lets latest-state reporting replace stale prior installation records.

Controls and important defaults

Monday 07:00 with StartWhenAvailable=true, so a missed trigger can run later. It runs directly instead of through the common launcher so its purpose-built Installed=false event remains authoritative. Deep Freeze log retention is owned centrally by Script 08.

How to verify the result

Filter Deep Freeze dashboard totals to Installed=true; inspect Unknown states and CLI errors. Thaw duration begins at observed tracking state, not necessarily the exact physical transition time. Confirm recently refrozen hosts publish a fresh state before treating the dashboard as current.

Deployment tool - Post-Deployment.ps1

Post-Deployment.ps1 | Version 2.3.0 | Source [S21]

Purpose and when it runs

A technician-invoked initial deployment workflow, not a recurring task in Register-Tasks_SYSTEM. It prepares a newly installed or refreshed endpoint and hands it over to the recurring maintenance package. Its public paths are template values and must be configured for the protected deployment environment.

How it works

1. Detect HP or Dell and prepare vendor tooling. Save power settings and temporarily remove idle power limits for long installation work.
2. Run vendor-aware updates with configured driver/software/BIOS/firmware behavior. Post-deployment includes BIOS/firmware by default, unlike the ordinary weekly Script 05 invocation; BitLocker is suspended when the selected vendor work requires it.
3. Install/verify PaperCut Print Deploy and Action1 when enabled. Installer timeout, MSI outcome, and missing-source errors are recorded independently so later sections can continue.
4. Safely refresh C:\Scripts from the configured share and run task registration. Preserve unrelated scheduled tasks.
5. Activate Office LTSC 2024 only if already installed and activation is enabled. Run shared Copilot removal, application updates, and Windows updates according to the enabled options.
6. Restore temporary power settings, clean working folders as permitted, summarize section failures/reboot requirements, and perform the configured final reboot.

Defaults and changes

InstallPaperCut, InstallAction1, RemoveMicrosoftCopilot, ActivateInstalledOffice, UpdateApplications, and InstallWindowsUpdates default true. IncludeSoftware=false, IncludeBiosAndFirmwareUpdates=true, RebootWhenComplete=true. Windows updates have a 90-minute timeout; PaperCut and Action1 each have a 900-second installer timeout. The shared Copilot module can be resolved from the central share even when the endpoint does not yet have C:\Scripts.

Evidence and technician handoff

Read each section's source/tool/installer results and the completed log in C:\Logs. Confirm the package copy, task principal/schedule, Office activation if applicable, management-agent state, final reboot, and restored power configuration. Continuing after a failure preserves later work; it does not convert the failed section into success. Confirm production template paths before use.

Support tools - launcher and task registration

Invoke-MaintenanceScript.ps1 v2.1.0

The launcher accepts an allowlisted ActionId, maps it to a fixed leaf-name script under C:\Scripts, and generates fixed child arguments. It rejects arbitrary paths/arguments; limited legacy translation accepts only exact approved combinations. It evaluates policy windows/dependencies, acquires the shared maintenance lock, supplies correlation context, launches Windows PowerShell, and publishes execution status. MicrosoftEdgeUpdateRepair has separately validated remediation metadata.

SystemRepairWeekly now fixes AutoRepairOnDetection, AllowComponentStoreEscalation, and AllowCopilotRemoval. Before launch, it verifies that Script 08 contains the approval integration; an older switch-only escalation script is refused. The launcher never retrieves the approval key on behalf of another unrelated action. Source: [S17].

Register-Tasks_SYSTEM.ps1 v4.12.0

Full reconciliation creates missing managed tasks and changes only tasks whose desired action/trigger/settings/principal differ. It removes known obsolete task names and references to retired scripts while preserving unrelated/Microsoft tasks. The desired task set includes the Sunday sequence, four-hour time sync, reboot startup-resume, and Monday Deep Freeze audit. The full run publishes task-reconciliation results and verifies the desired tasks exist.

The new targeted mode requires an approval-integrated Script 08 and exactly one existing 11. System Repair or 12. System Repair task running as SYSTEM. It replaces only the action with a direct Script 08 invocation and preserves scheduling/principal/settings. It does not start the task, create a missing one, or remove obsolete tasks. Source: [S23].

```
Set-Location C:\Scripts
.\Register-Tasks_SYSTEM.ps1 -SystemRepairOnly -WhatIf
.\Register-Tasks_SYSTEM.ps1 -SystemRepairOnly
```

Deploy the merged registration and launcher together with Script 08 and the matching manifest. The older endpoint-tested registration copy was not substituted for the newer GitHub reconciliation implementation. The merged versions still need final Windows parser/task validation.

Support modules - shared framework and Copilot

Maintenance.Framework.psm1 v2.4.0

Provides shared paths/run context, privilege checks, policy loading, maintenance-window and dependency evaluation, execution locking, common logging, atomic/latest JSON support, NDJSON writes/rotation, and fleet-status publishing. Telemetry writes use synchronization to reduce file-lock collisions. Active text logs are staged outside the watched final-log location, then published as completed files. This prevents log collection from consuming a partial execution record.

Framework defaults include a 50 MB telemetry rotation threshold, 60-day retention, and the C:\Temp\Compton-Maintenance-Logs staging root; individual scripts can override staging, particularly Script 08 and Script 10 beneath ProgramData. The policy also contains log settings, but the effective value depends on which function/caller passes it. Do not assume a single global retention setting controls every text, JSON, rollback, and inventory artifact. Source: [S19].

Maintenance.Copilot.psm1 v1.2.1

The canonical removal implementation is shared by Script 04, Script 08 when explicitly enabled, and post-deployment. It removes targeted Copilot AppX registrations and online provisioning, handles the Microsoft 365 Copilot package Microsoft.MicrosoftOfficeHub and explicit Windows AI provider scope, stops Copilot processes/tasks/startup entries, and applies machine/current/future-user suppression settings. It also processes narrowly identified registered desktop Copilot uninstallers, validates outcomes, and removes verified orphaned application remnants.

It reports found/attempted/removed/remaining package and desktop results, uninstall exits, and failures. Removal is targeted rather than a general uninstall of all Microsoft Office applications or hardware utilities. Policy and cleanup results can be affected by GPO or later application servicing; Script 04 and the enabled Script 08 fallback provide repeat enforcement. Source: [S18].

Cross-script ownership

Responsibility	Owner
Recurring application/configuration sections	Script 04
Shared Copilot implementation	Maintenance.Copilot.psm1
Fleet cleanup/log retention	Script 08
Reboot cycle coordination	Script 07 (Setup also has its own repair reboot)
Current endpoint/software evidence	Script 14
Deep Freeze observed state	Script 16

Policies, manifests, and administrative utilities

Maintenance.Policy.json and SoftwareInventory.Policy.json

Maintenance policy defines script windows, dependency requirements, lock timeout, stale-fleet thresholds, and fleet-status destinations. The reviewed defaults use warning after three days and critical after seven for fleet-status age, a 120-second execution-lock timeout, and template share roots. Missing script entries are allowed by the current window function, so completeness matters. Retired-name entries still appear in the policy; they do not create tasks by themselves.

Software inventory policy v1.0.0 tracks non-system products, excludes OS updates/hotfixes and common runtime/system components, normalizes categories/names, and retains local software snapshot history for 730 days where the caller uses that setting. IncludePerUserRegistryForLoadedProfiles=false limits the default scope. Its exclusion list is part of interpreting inventory gaps, not evidence an application was uninstalled. Sources: [S20], [S26].

DeploymentManifest.json / SHA256SUMS.txt / BUILD-VALIDATION.json

The manifest describes approved managed filenames, roles, embedded versions, and SHA-256 hashes. The checksum list supports independent byte verification. BUILD-VALIDATION records structural checks and distinguishes performed checks from required Windows parser testing. Package 3.0.3 includes Script 08 4.9.2, registration 4.12.0, launcher 2.1.0, and the previously approved helpers. A matching checksum establishes file identity, not correct runtime behavior.

Update-DeploymentManifest.ps1 v2.3.0

Reads the source-share files, adds missing approved catalog entries, retires known replaced names only when their replacement is ready, and refreshes versions/hashes for existing manifest entries. It backs up and validates the manifest before safe replacement, with WhatIfOnly preview support. Its catalog currently lacks Repair-Windows-ComponentStore.ps1 and SoftwareInventory.Policy.json. Existing entries can still be processed, but missing entries will not be automatically added by that catalog; align it before using regeneration as a complete package check. Source: [S29].

Get-MaintenanceFleetStatus.ps1 v1.0.0

Manually reads JSON status files from the configured status share and writes an HTML fleet report, marking stale data using warning/critical-day thresholds. It reports file-based fleet status rather than directly querying Elastic, repairing endpoints, or creating scheduled tasks. Production paths must replace templates. Source: [S16].

Validation tools and conditional Edge repair

Test-AllMaintenanceScripts.ps1

Runs the PowerShell language parser over .ps1/.psm1 files directly in the selected source folder, listing syntax errors with location/context and exiting nonzero on errors. It does not run those maintenance scripts, validate credentials, test registry/service behavior, or prove a successful repair. Use Windows PowerShell 5.1 to match the production target. It is a controlled administrative check, not a recurring maintenance task. Source: [S27].

Test-MaintenanceTelemetry.ps1

Exercises synchronized NDJSON writing/rotation using six worker processes with 100 records each by default. It uses a dedicated Maintenance-Telemetry-Test.ndjson and test mutex, then checks JSON parsing and archives. It is suitable when the framework telemetry writer changes; it is not a routine diagnostic to launch on every endpoint. Source: [S28].

Repair-MicrosoftEdgeUpdate.ps1 v1.0.0

A detection-first conditional remediation helper for the MicrosoftEdgeUpdateRepair class. It evaluates Edge installation, update policy, services, and tasks; repairs supported service start/running state and enables eligible updater tasks. It can invoke the installed updater, wait, and verify the final servicing state. If configured/necessary, it resolves an approved Enterprise MSI and performs a validated installer repair.

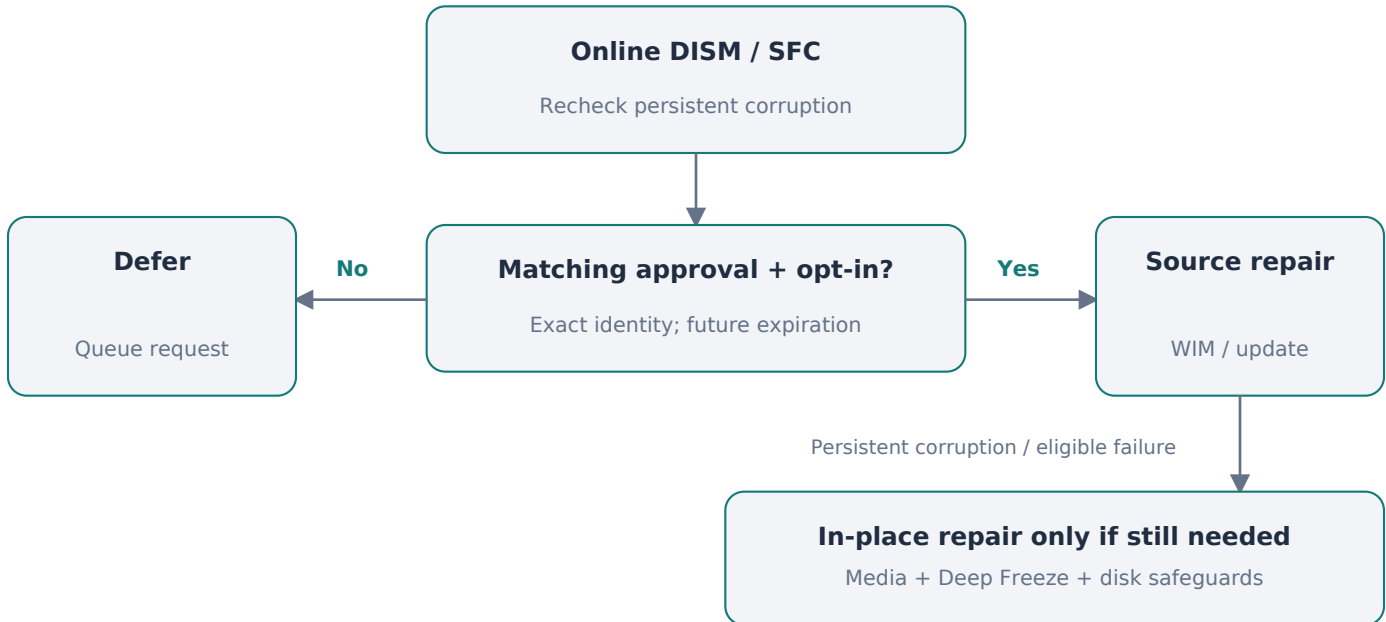
CorrelationId, FindingFingerprint, and AttemptNumber connect the result to a controlled remediation request. Timeouts bound updater/installer operations. The launcher accepts remediation metadata for this action only. The helper is not registered as another Sunday Edge task in this package. Source: [S24].

Scope and verification

Edge servicing repair does not broadly alter favorites, profiles, browser user data, or lab homepage policies. Those configuration responsibilities remain in Script 04. Verify before/after updater services/tasks, policy blocks, trusted installer evidence, action list, repeat-detection fingerprint, and final result. A blocked update policy or missing trusted installer is a recorded condition, not an instruction to bypass organizational policy.

Inventory can emit candidate findings, but this guide does not claim that every candidate class has a deployed automated n8n dispatcher. The newly demonstrated workflow covers Script 08 repair approval; other remediation orchestration requires separate evidence.

In-place repair - management decision model



Why approval exists

Ordinary Windows servicing repair is less disruptive than a Windows Setup repair upgrade. When DISM/SFC cannot resolve corruption, an administrator must authorize the endpoint-specific escalation. The request includes the computer, reason, originating version/run, and a fingerprint representing the observed corruption signals. A technician can approve or reject a pending request without manually typing a repair key on the workstation.

What an approval authorizes

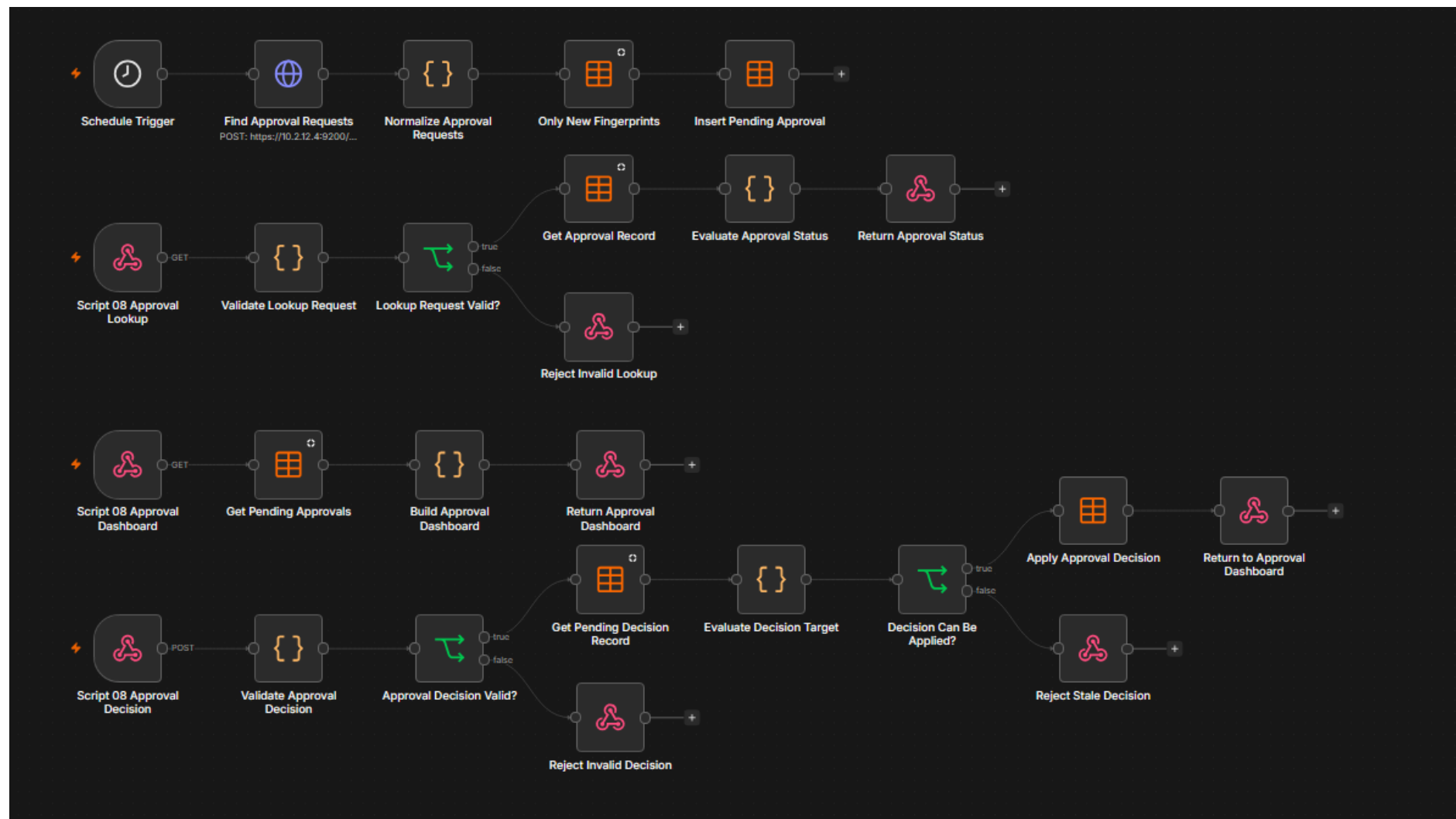
A matching, unexpired decision allows Script 08 to invoke the component-store helper while explicit escalation is enabled. The helper first tries repair sources and verifies the result. If that succeeds, no in-place install is needed. If escalation remains necessary and safeguards pass, Windows Setup runs in upgrade mode to retain applications, files, profiles, domain membership, and settings. Compatible newer media can also change the Windows build; this is not a wipe-and-reimage operation or a guarantee that every application will remain healthy.

Timing and service impact

Approval does not reserve a precise repair duration or guarantee the next Sunday succeeds. The computer must execute the task, still exhibit the approved signature, reach n8n and the share, and satisfy the helper prerequisites before expiration. Windows Setup can take longer than the normal repair slot and reboot the machine. Plan a maintenance window and avoid returning the endpoint to classroom use until post-repair verification is complete.

n8n workflow - supplied implementation image

Four branches: scheduled collection, endpoint lookup, technician dashboard, and decision handling. The exact polling interval and every live node setting require review in n8n.



The canvas is a reference image of the supplied workflow, not an exported workflow JSON. Preserve/backup the live workflow separately. Authentication credential values are intentionally omitted from this guide.

n8n - branch-by-branch technical flow

1. Scheduled collection

Schedule Trigger starts Find Approval Requests, an authenticated Elastic search against logs-compton.maintenance-*. The request selects endpoint.remediation.approval_required on maintenance_event.EventType, an exact keyword field, within now-30d; results sort @timestamp descending with size=100. Normalize Approval Requests reads hits.hits / _source.maintenance_event, trims identity, ignores missing identity, and retains the first/newest hit per fingerprint within the returned batch.

Normalized rows start Pending, NotRun, and corruptionStillPresent=true. requestedAt comes from the source timestamp; expiresAt is 14 UTC days later. Only New Fingerprints checks absence across all statuses before Insert Pending Approval. Do not filter this existence check to Pending or approved/rejected rows can be reinserted. Keep Always Output Data off on that insertion filter. A high event rate can exceed the 100-event window; schedule frequency/query size/pagination must be sized for the real backlog.

2. Endpoint lookup

Script 08 Approval Lookup receives a GET with Header Auth. Validate Lookup Request checks normalized computer/fingerprint input. Lookup Request Valid routes invalid input to rejection; valid input queries Get Approval Record, evaluates the row, and returns JSON. Missing/expired/non-approved rows must return approved=false. A successful HTTP lookup and a granted approval are different outcomes.

3. Dashboard rendering

The Basic Auth GET dashboard selects Pending rows, builds escaped HTML forms, and returns Text with status 200 and Content-Type: text/html; charset=utf-8. Get Pending Approvals uses Always Output Data so no rows still reach the builder; the empty placeholder item is ignored. A valid empty state is a rendered page reporting zero pending requests, not a blank response.

4. Technician decision

The Basic Auth POST accepts fingerprint, computerName, decidedBy, and Approved/Rejected. The validation branch checks identity, decision, technician-name length, and expected-host/referrer rules. Get Pending Decision Record and Evaluate Decision Target require a matching pending, unexpired target. Apply Approval Decision updates only requestStatus, decisionAt, and decidedBy; stale/invalid branches reject. Return to Approval Dashboard sends a 303 redirect to the matching DNS-host dashboard URL.

Approval table - identity, state, and expiry

Every data-table node must select script08_repair_approvals. Match decisions by fingerprint and computer identity with Pending status. The fingerprint already incorporates the uppercase computer name, but the endpoint also checks the explicit returned computer field.

Fields	Purpose
fingerprint, computerName, runId	Repair identity and originating execution; 64 lowercase hex fingerprint and uppercase computer.
requestStatus	Pending, Approved, or Rejected decision state; expired approval is refused regardless of the displayed stored state.
requestedAt, lastSeenAt, expiresAt	Request time, initial last-seen value, and expiry 14 days after request. Repeated events are not an automatic renewal.
decisionAt, decidedBy	UTC decision time and submitted technician name. Name is audit text, not independent proof of a unique signed-in identity.
reason, scriptVersion	Displayed evidence and originating script version.
repairOutcome, failureDetail, corruptionStillPresent, consumedAt	Lifecycle fields initialized for requests; the demonstrated workflow does not synchronize final repair results or consume approval.
id, createdAt, updatedAt	n8n-managed row metadata.

Lifecycle and deduplication

A new identity becomes Pending. Approval/rejection changes the existing row, and an Approved row disappears from the pending-only dashboard. It remains available for lookup until its expiresAt. Repeated collection of the same fingerprint is skipped across every status. Expired/rejected rows do not automatically become fresh pending requests; an administrator must deliberately manage reapproval. Expired Pending rows can remain displayed, but target/endpoint expiration checks should refuse them.

Current boundaries

Approval is not single use in the shown implementation. consumedAt is not set by Script 08, and repair-result telemetry is not ingested back into these rows by the supplied four branches. Do not use NotRun or corruptionStillPresent=true in an old approved row as authoritative final health. Conversely, do not set corruptionStillPresent=false just because a technician approved the work. Look to post-repair logs and fresh endpoint evidence.

The existence filter plus insertion is not a demonstrated database uniqueness constraint. Avoid overlapping collector executions or enforce uniqueness/transactional handling if concurrent inserts become possible. This is an implementation limit to review, not a claim that duplicate requests occurred.

Approval API and endpoint authorization contract

Method / route	Authentication and purpose
GET /webhook/script08/approvals	Administrator Basic Auth. HTML pending-request dashboard.
POST /webhook/script08/approval-decision	Same Basic Auth. Validate and apply a technician decision.
GET /webhook/script08/approval-status	Header Auth: x-compton-remediation-key. Endpoint status lookup.

Current production base: <http://n8n.compton.edu:5678>. Published production paths use /webhook/. Test paths use /webhook-test/ and require an active test execution/listener. Forms and redirect targets must use the same DNS host and matching mode; an internal-IP form action mixed with a DNS dashboard caused origin/404 troubleshooting during setup.

Key retrieval and request checks

Script 08 reads and trims \\filesvr\labscripts\Installers\approval-key.txt only when approval lookup is needed. It neither creates the file nor sets permissions. The administrator manages access; SYSTEM runs must be tested using the endpoint/computer account's effective share access. The key value must match the n8n Header Auth credential and must not appear in logs, screenshots, or repository configuration.

The configured lookup URL must be an absolute HTTP/HTTPS production approval-status endpoint with no existing query, fragment, or embedded credentials. The script URL-encodes computerName/fingerprint, applies a default 15-second timeout, and refuses redirects. HTTP is the current protected-network configuration; an HTTPS endpoint can be configured without embedding credentials in the URL.

All authorization conditions must pass

- Response approved is a native Boolean true, not the string "true".
- Status exactly equals Approved.
- Returned computer name and fingerprint exactly match the current request.
- expiresAt parses successfully and is later than current UTC time.
- AllowComponentStoreEscalation is enabled, and corruption remains after ordinary repair.

Missing key, access failure, unreachable API, malformed response, expiry, rejection, or identity mismatch defers escalation. NotFound is a successful lookup with no record and no authorization. The sandboxed n8n HTML may send Origin:null; retain the expected-origin/referrer validation rather than removing it.

Component-store helper - stages and safeguards

Repair-Windows-ComponentStore.ps1 | Version 2.2.0 | Source [S25]

Before Windows Setup

1. Script 08 confirms the helper, install.wim, and expanded setup.exe are available. It copies the helper locally under ProgramData and compares its SHA-256 with the source.
2. The helper verifies Administrator rights and Deep Freeze state: NotInstalled or Thawed can proceed; Frozen or unverifiable installed state is blocked.
3. Match the installed edition, architecture, and language to WIM metadata. Stage/select a compatible ADK DISM when required, then run CheckHealth and ScanHealth.
4. Try RestoreHealth with wim:path:index and LimitAccess. For eligible missing-content errors, permit Windows Update as an additional source unless explicitly disabled; run SFC and final DISM assessment.
5. If corruption/eligible repair failure remains, validate setup media and free space, stage it locally, and verify required setup-file hashes before launching the repair upgrade.

In-place setup requirements and changes

Default MinimumFreeSpaceGB is 40; the helper also considers source-media size and headroom. Expanded media must contain the required setup files and a suitable image. Older media is refused for in-place repair. The local setup stage defaults to C:\Temp\Compton-Windows11-25H2-Repair. Windows Setup is launched in automated upgrade mode, intended to preserve installed applications and user state. The source-media build may change the running Windows version.

The helper records and disables eligible currently enabled Compton maintenance tasks so other jobs do not interrupt Setup. It persists phase/boot/task state, registers Compton - Complete Component Store Repair, and suspends BitLocker for three reboots when needed. Failures before accepted setup invoke applicable restoration handling. Review exact logs when failures leave a pending repair state.

The approval gate is in Script 08. Directly invoking this helper is an administrative entry point and does not independently query n8n. Its EnableInPlaceRepair parameter defaults true. Restrict who can run or modify the helper; use the approved Script 08 path for the managed workflow.

Post-Setup verification and return to service

Persistent state prevents immediate relaunch

The helper records the launch boot timestamp and InPlaceRepairLaunched phase. If invoked again during the same Windows boot, it refuses to launch another repair cycle and exits with reboot-required status. After a newer boot, it checks whether Setup-related processes are still active and defers verification if they are. This distinguishes accepted Setup from a verified completed reboot.

Verification task

Compton - Complete Component Store Repair runs as SYSTEM. It has a startup trigger delayed 20 minutes and another trigger beginning one hour after registration and repeating hourly for one day. It uses StartWhenAvailable, a six-hour execution limit, ignore-new-instance handling, and two restart attempts at 15-minute intervals. The task reruns the staged helper to evaluate pending state; verify these settings in the deployed task export.

Verification and cleanup

Post-upgrade verification runs DISM ScanHealth and SFC unless explicitly skipped. Command failure records PostRepairVerificationFailed and requires follow-up; it is not a successful repair. On accepted verification, the helper restores the maintenance tasks it previously recorded as enabled, removes the resume task/state, and removes the guarded local setup stage unless KeepLocalSetupMedia is set. It also cleans its staged ADK artifacts as applicable.

Operators should inspect command text, exit codes, fresh Script 08/14 evidence, installed Windows version, BitLocker protection, Deep Freeze state, and key applications. The helper's post-boot command-success checks and cleanup are automation evidence, but they do not independently test every application, domain operation, or policy setting. Verify the actual healthy DISM/SFC output before closure.

Return-to-service conditions

- Confirm the new boot and completed Setup; preserve failure/setup logs when escalation did not complete.
- Confirm clean servicing/system-file evidence, expected domain access, and working classroom applications and printing.
- Verify originally enabled maintenance tasks were restored, no repair-resume task remains unexpectedly, and BitLocker protection is active as required.
- Keep the workstation thawed through validation where Deep Freeze is installed; refreeze through the approved operational procedure after confirming the repair persists.

Technician runbook - review and approve

Request review

1. Open the production Script 08 Repair Approvals dashboard on the DNS host and authenticate with the approved administrator credential.
2. Verify the computer identity, current request reason, timestamp, expiry, and fingerprint. Locate its latest Script 08 diagnostic evidence in Elastic/local logs.
3. Confirm the computer remains eligible for a maintenance window, the media/helper shares are accessible under SYSTEM, Deep Freeze can be thawed, and sufficient disk space is available.
4. Enter the technician name and approve only the intended pending identity, or reject it with follow-up handled through the established ticket process.
5. Confirm the decision row and API result. An approved request disappearing from the pending dashboard is expected; it does not mean the repair completed.

Expected next run

The next eligible Script 08 execution runs normal diagnostics/repair again. If the machine is now clean, escalation is unnecessary. If the signal changes, the old approval does not authorize the new fingerprint. If the approved signature persists, expiry and response identity must still validate. Source repair may resolve the issue without Setup; otherwise the helper assesses Setup safeguards.

Simple interpretation examples

Observed result	Meaning / follow-up
LookupSucceeded=true; Approved=false; NotFound	Connectivity/authentication worked; no record grants authorization. Confirm collection/identity.
DeferredAwaitingApproval	Normal corruption escalation was not authorized. Review dashboard/key/API/expiry/flags.
Approved / ApprovedByN8n	Matching unexpired permission exists; not proof the helper ran.
StartedRebootRequired	Setup handoff requires later boot/verification. Keep endpoint out of service until validated.
PostRepairVerificationFailed	Escalation cannot be closed as repaired. Preserve logs and investigate retained state/tasks.

Do not create a production approval for the synthetic all-zero fingerprint merely to prove a real endpoint can install Windows. Use the endpoint's current fingerprint and a controlled pilot when functional escalation testing is authorized.

Technical runbook - deployment and validation

Publish code versus deploy code

GitHub main commit c82b57e records the approved source update. Deploy the tested Script 08, launcher, registration, modules/policy/helpers, and byte-matching manifest to the protected central share before endpoint distribution. Do not regenerate production hashes from differently configured public-template files. The updater consumes the deployment share, not a GitHub webhook. Back up/export the live n8n workflow and its table separately; the repository contains the workflow image and instructions, not an exported workflow definition.

Controlled validation sequence

1. Configure required template sources/credentials in the protected environment. Keep the approval key external and manage its access separately.
2. Parse the final package under 64-bit Windows PowerShell 5.1. Check manifest versions/hashes and align the regeneration catalog with the complete approved file set.
3. Pilot share synchronization on one endpoint. Verify copy/rollback/reconciliation status, the installed action/principal/triggers, and selected mode.
4. Verify key-file/API access under SYSTEM without printing the key. Use a synthetic request for dashboard/decision/lookup checks; remove test rows afterward.
5. Pilot the real corruption lifecycle during an approved maintenance window: ordinary repair, matching approval, source repair, conditional Setup, newer boot, verification, and return-to-service checks.
6. Compare fresh telemetry and local logs, then broaden deployment only after the pilot outcomes and timing are understood.

```
powershell.exe -NoProfile -File C:\Scripts\Test-AllMaintenanceScripts.ps1
# Preview an existing repair-action update only:
.\Register-Tasks_SYSTEM.ps1 -SystemRepairOnly -WhatIf
```

Known baseline items to verify

Script 04's Elastic installation switch is false; enable/configure it only as intended in the protected deployment. Its current policy entry is absent, so a scheduled time alone is not a policy guard. Profile cleanup defaults to no age threshold. The manifest utility's catalog is incomplete for two newer managed files. The n8n collector interval, credentials, row-uniqueness handling, approval renewal, consumption, and result-to-table synchronization need explicit operational ownership.

Troubleshooting the approval workflow

Symptom	Likely checks
Blank dashboard	Trace the complete GET execution through Get Pending Approvals, HTML builder, and Respond to Webhook. Use Text / \$json.html / 200 / HTML content type. Ensure zero rows still reach the builder.
404 after approve	Verify workflow is published or test listener is active; POST path, absolute action host, and 303 redirect must match execution mode.
Decision routed false	Inspect validation outputs and target status: valid identity, Approved/Rejected, 2-100-character name, trusted host/referrer, pending record, future expiry.
No collector rows	Check exact EventType keyword term, index/data-stream pattern, 30-day time range, actual ingestion, result count, and pinned mock data.
ApprovalLookupUnavailable	Check DNS/network, endpoint route, key contents versus Header Auth, SYSTEM share/file access, timeout, and refused redirects. Do not expose credentials in diagnostics.
NotFound	Lookup succeeded but no exact computer/fingerprint record exists. Check collection and current signature.
Approved but no install	Check task actually ran, corruption still exists, same signature/expiry/opt-in, and helper prerequisites. Source repair may have succeeded.
Pending row after expiry	Dashboard filters Pending; expired rows may remain. Target/API expiry checks still refuse authorization. Manage deliberate renewal separately.
Repair state persists / tasks disabled	Review helper phase, boot proof, Setup processes, verification logs and task restoration. Do not erase state before understanding the failure.

Evidence priority

Start with the current execution's RunId, task result, and time. Then inspect the failing node/step and its actual inputs, followed by table state and endpoint/helper logs. Old pinned editor data or an earlier Approved response can mislead when the current record has been deleted, expired, or replaced by a different signature.

Telemetry, reporting, and operating ownership

What staff can measure

Question	Evidence
Did maintenance execute recently?	Task history/result, maintenance.execution time/RunId, fleet status and snapshot freshness.
What updated or failed?	Per-application/driver/Windows-update results, installed versions, remaining candidates and failure details.
Which settings changed?	Section outcomes and before/after registry/service/task state; verified Office, printer, policy and startup evidence.
Which endpoints need attention?	Fresh endpoint.health findings, severity, failed collector context, storage/security/agent evidence.
Was repair authorized and completed?	n8n row/decision plus exact fingerprint; Script 08 result; helper Setup/boot/verification logs and post-repair health.

Storage and ingestion

Most scripts publish completed text/YAML/latest JSON plus NDJSON under C:\Logs, with working logs staged elsewhere. Script 16 also writes atomic Deep Freeze JSON events. Elastic pipelines place event fields beneath maintenance_event; Script 08 collection uses that namespace. A latest-state dashboard must use the newest relevant observation and account for explicit removal/NotInstalled evidence. Log rotation and retention affect local history; they are not proof of centralized archive retention.

Practical responsibility model

Systems engineering: controlled sources, media compatibility, task/package distribution, n8n availability and backups. **Desktop support:** request review, endpoint preparation, pilot validation, and return-to-service checks. **Network/security staff:** access/connectivity and credential/authorization controls, ingestion health, and repeat-failure analysis. **Management:** maintenance-window/service-impact decisions and outcome reporting. These are suggested ownership areas, not a newly imposed organizational approval policy.

Closure standard

Close a maintenance or repair issue using fresh verified outcome evidence. A completed command, granted approval, or no pending dashboard row is not enough by itself. Distinguish failure, expected skip, deferred work, reboot-required handoff, and a verified healthy endpoint in reports.

Retired scripts and replacement ownership

The Retired folder preserves history. These files are not desired active tasks or managed deployment entries. Script numbering is historical: a task number does not always equal its script number, and gaps do not mean a missing scheduled stage.

Retired script	Current replacement / ownership
04_Update_Edge_Silent.ps1	Repair-MicrosoftEdgeUpdate conditional helper plus Script 14 evidence; no standalone recurring Edge task.
11_Install_SharpDriver_And_PaperCut.ps1	Script 04 printing/PaperCut section.
12_Enable-SystemRestore-And-Create-RestorePoint.ps1	Script 04 System Restore section.
13_Configure_Autologon_And_Edge.ps1	Script 04 targeted Edge InPrivate; autologon change disabled.
15_Install_Elastic_Agent.ps1	Script 04 embedded Elastic section; currently disabled in reviewed GitHub source.
17_Set_Browser_Homepage.ps1	Script 04 browser policies.
18_Install_Honorlock_Chrome_Extension.ps1	Group Policy; embedded Script 04 section retained but disabled.
19_Stellarium_Location_Services.ps1	Script 04 targeted location settings.

Why consolidation matters

A single owner for each recurring configuration avoids multiple tasks changing the same settings in different orders. Script 00 and registration remove recognized obsolete files/task names only with their replacement safeguards. Review legacy tasks on endpoints with older deployments, especially when using targeted repair updates, since that mode intentionally leaves the rest of the schedule intact.

Outside this baseline

Older menus, custom compliance-audit scripts, installation-media/domain-join utilities, and external reporting workflows may exist in the environment but are not part of the reviewed package's active catalog. They should receive separate documentation from their actual current source rather than inheriting behavior or schedules from historical conversations.

Glossary and interpretation guide

Term	Meaning in this program
SYSTEM	Windows service account used by scheduled tasks. Share access uses the endpoint's effective computer/service identity, not the technician's interactive session.
DISM	Windows servicing tool for assessing/repairing the component store used by system updates and feature servicing.
SFC / CBS	System File Checker validates/protects system files; Component-Based Servicing logs contain diagnostic repair evidence.
WIM / ADK	Windows image container used as a repair source; Assessment and Deployment Kit supplies servicing tools where needed.
In-place repair	Windows Setup upgrade-style repair designed to retain applications and user state; may update the Windows build with compatible media.
Fingerprint	Stable SHA-256 identifier for computer/corruption evidence. It links a request and approval to a specific observed signature.
RunId / correlation ID	Execution identifiers used to connect logs/events. Unlike fingerprints, they identify runs rather than persistent findings.
NDJSON / latest snapshot	One JSON object per line for event ingestion / a current snapshot file. Snapshot absence or age must be considered.
Tombstone	Explicit removal event that lets software latest-state reporting remove a previously seen package.
Deep Freeze / Thawed	Endpoint protection state that may discard changes when frozen; repairs require the appropriate thawed state and verification.
Pending / Approved / NotFound	Undecided table request / granted decision subject to identity and expiry / successful lookup with no matching record.
Verified / deferred / skipped	Observed successful result / work postponed for an unmet condition / work intentionally not performed. These are not interchangeable.

Source baseline and validation record

This guide was prepared from the actual repository source at [commit c82b57ef30219b9a01d92a2586756105fa5c58bc](#), package 3.0.3, October 8, 2026. Source code takes precedence over older README feature descriptions. The supplied n8n screenshot and the October 8 test outputs establish the workflow structure and demonstrated responses; no live workflow export or fleet task inventory was supplied.

Verified during preparation

- Reviewed current script headers, parameters, execution sections, registration times/settings, policy behavior, helper safeguards, and active manifest versions.
- Confirmed the eight GitHub update entries match their prepared blobs and main points to the published commit.
- Used the demonstrated synthetic request/decision/lookup and SYSTEM-context results for the stated test evidence.
- Generated and rendered the PDF for layout review; no endpoint maintenance or Windows Setup was executed while preparing this guide.

Not yet established by this document

- Final Windows PowerShell 5.1 parser/runtime validation for the merged launcher and registration.
- A completed real persistent-corruption -> approval -> source repair -> Setup -> post-boot verification lifecycle.
- Uniform deployment-share configuration, installed endpoint schedules/versions, live n8n polling interval, complete authorization audit identity, or automatic table outcome synchronization.

Source references

The following inventory provides exact source-file references for the script chapters. Each filename links to its immutable version at the reviewed commit. Administrative utility files without an embedded version are identified as such. The workflow image is sourced from the user-provided October 8 canvas.

Source inventory - numbered and deployment scripts

Ref	Source file	Version
[S01]	00_Update-Scripts-FromShare.ps1	4.11.2
[S02]	01_Enable_Windows_Update_Services.ps1	2.8.5
[S03]	02_Remove_User_Profiles.ps1	2.5.0
[S04]	03_Weekend_Apps_Update.ps1	2.2.0
[S05]	04_Sunday_Lab_Application_Maintenance.ps1	1.17.0
[S06]	05_Weekend_HP_Drivers_Update.ps1	3.4.3
[S07]	06_Weekend_Windows_Updates.ps1	2.1.10
[S08]	07_Force_Reboot_Install_Updates.ps1	2.1.1
[S09]	08_System_Repair.ps1	4.9.2
[S10]	09_Disable_Windows_Update_Services.ps1	2.0.7
[S11]	10_Sync_System_Time.ps1	2.0.10
[S12]	14_Endpoint_Health_Inventory.ps1	1.9.0
[S13]	16_Check_Deep_Freeze_Status.ps1	1.3.0
[S21]	Post-Deployment.ps1	2.3.0

Each source filename above links directly to the reviewed version at commit c82b57e. Scripts are described from this source baseline; compare with the protected-share copy before operational use.

Source inventory - support and reference files

[S14] BUILD-VALIDATION.json

[S15] DeploymentManifest.json

[S16] Get-MaintenanceFleetStatus.ps1

[S17] Invoke-MaintenanceScript.ps1

[S18] Maintenance.Copilot.psm1

[S19] Maintenance.Framework.psm1

[S20] Maintenance.Policy.json

[S22] README.md

[S23] Register-Tasks_SYSTEM.ps1

[S24] Repair-MicrosoftEdgeUpdate.ps1

[S25] Repair-Windows-ComponentStore.ps1

[S26] SoftwareInventory.Policy.json

[S27] Test-AllMaintenanceScripts.ps1

[S28] Test-MaintenanceTelemetry.ps1

[S29] Update-DeploymentManifest.ps1

Checksum reference: SHA256SUMS.txt

Workflow image: script08-repair-approval-workflow.png

Refresh this guide when the protected deployment configuration, scheduled tasks, approval contract, media version, or active source baseline changes. Preserve the distinction between reviewed code, deployed configuration, and demonstrated operational results.